1. L1 Norm (Norm-1 / Manhattan Norm)

Definition: The **L1 Norm** is the sum of the absolute values of all elements in a vector.

Formula

$\|x\|_1 = \Sigma|x_i|$

Characteristics

- Sum of absolute values.
- Also known as **Manhattan Norm** or **Taxicab Norm**.
- Less sensitive to outliers.
- Encourages sparse solutions (many coefficients become zero).

2. L2 Norm (Norm-2 / Euclidean Norm)

Definition: The **L2 Norm** is the square root of the sum of the squared values of all elements in a vector.

Formula

$\|x\|_2 = \sqrt{(\Sigma x_i^2)}$

Characteristics

- Measures the straight-line (Euclidean) distance.
- Also known as the **Euclidean Norm**.
- Sensitive to outliers because larger values have a greater impact.
- Does not encourage sparse solutions.
- Smooth and differentiable, making it suitable for optimization algorithms.

Dot Product and Angle Between Two Vectors

1. Dot Product (Scalar Product)

Definition

The **Dot Product** (also called the **Scalar Product**) of two vectors is a mathematical operation that multiplies corresponding components of the vectors and adds the results. The output is always a **scalar (single number)**.

Formula

$$\mathbf{A \cdot B = A_x B_x + A_y B_y + A_z B_z}$$

or

$$\mathbf{A \cdot B = \|A\| \, \|B\| \cos \theta}$$

Characteristics

- Produces a **scalar value**.
- Depends on both the magnitudes of the vectors and the angle between them.
- Measures how much one vector points in the direction of another.
- Used to determine whether vectors are perpendicular.

---

2. Angle Between Two Vectors

Definition

The **angle between two vectors** measures how closely they point in the same direction. It is calculated using the **Dot Product**.

Formula

$$\mathbf{\cos \theta = (A \cdot B) / (\|A\| \, \|B\|)}$$

Therefore,

$$\mathbf{\theta = \cos^{-1} ((A \cdot B) / (\|A\| \, \|B\|))}$$

Characteristics

- The angle always lies between **0° and 180°**.

- Depends on both the dot product and the magnitudes of the vectors.
- Used to measure the similarity in vector directions.

Special Cases

Angle	Meaning
0°	Vectors point in the same direction.
90°	Vectors are perpendicular (Orthogonal).
180°	Vectors point in opposite directions.

Relationship Between Dot Product and Angle

- **Dot Product > 0** → Angle **< 90°**
- **Dot Product = 0** → Angle **= 90°**
- **Dot Product < 0** → Angle **> 90°**

Vector Projection

Definition

The **Vector Projection** gives both the **magnitude and direction** of the projected vector.

Formula

$$\mathbf{proj_B(A) = (A \cdot B / \|B\|^2) B}$$

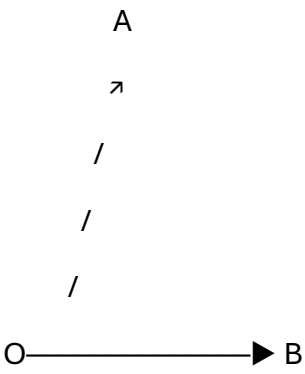
Characteristics

✓ Produces a **vector**.

✓ Has the same direction as vector **B**.

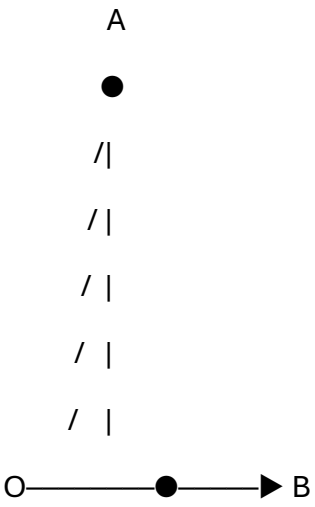
✓ Represents the part of vector **A** that acts along **B**.

Visual Representation



Projection of A onto B

Or



Projection

Interpretation

- **Large Projection** → Most of vector **A** points in the direction of **B**.
- **Zero Projection** → Vectors are perpendicular.
- **Negative Projection** → Vector **A** points opposite to vector **B**.

Matrix Addition and Multiplication

---

1. Matrix Addition

Definition

Matrix Addition is the process of adding two matrices by adding their corresponding elements.

Note: Matrix addition is possible **only if both matrices have the same dimensions (same number of rows and columns).**

Formula

If

$\mathbf{A = [a_{ij}]}$ and $\mathbf{B = [b_{ij}]}$

then

$\mathbf{A + B = [a_{ij} + b_{ij}]}$

Characteristics

- Both matrices must have the **same dimensions**.
- Add corresponding elements.
- The resulting matrix has the **same dimensions**.

Applications

- Image Processing
- Data Analysis
- Computer Graphics
- Engineering Calculations

---

2. Matrix Multiplication

Definition

Matrix Multiplication combines two matrices by multiplying the rows of the first matrix with the columns of the second matrix.

Condition: Matrix multiplication is possible only when

Number of columns of A = Number of rows of B

If

$\mathbf{A_{mxn}}$ and $\mathbf{B_{nxp}}$

then

$\mathbf{AB = C_{mxp}}$

Formula

$\mathbf{(AB)_{ij} = \Sigma\, a_{ik}\, b_{kj}}$

Characteristics

- Number of columns of the first matrix must equal the number of rows of the second matrix.
- Multiply rows by columns.
- The order of multiplication matters.

Applications

- Machine Learning
- Neural Networks
- Computer Graphics
- Robotics
- Image Processing

- Data Transformations

Matrix Transpose and Inverse

---

1. Matrix Transpose

Definition

The **transpose of a matrix** is obtained by interchanging its **rows and columns**. The transpose of a matrix **A** is denoted by $\mathbf{A^T}$.

Formula

If

$\mathbf{A = [a_{ij}]}$

then

$\mathbf{A^T = [a_{ji}]}$

Characteristics

- Rows become columns.
- Columns become rows.
- The dimensions are reversed.
- If **A** is an **m × n** matrix, then $\mathbf{A^T}$ is an **n × m** matrix.

Applications

- Machine Learning
- Data Analysis

- Computer Graphics
- Statistics
- Linear Algebra

---

2. Matrix Inverse

Definition

The **inverse of a square matrix** is another matrix that, when multiplied by the original matrix, produces the **Identity Matrix**.

The inverse of matrix **A** is denoted by $\mathbf{A^{-1}}$.

Formula

$$\mathbf{AA^{-1} = A^{-1}A = I}$$

where **I** is the **Identity Matrix**.

For a **2 × 2** matrix

$$\mathbf{A} = \begin{matrix} \mathbf{a} & \mathbf{b} \\ c & d \end{matrix}$$

the inverse is

$$\mathbf{A^{-1} = (1 / (ad - bc))} \begin{matrix} \mathbf{d} & \mathbf{-b} \\ -c & a \end{matrix}$$

provided that

$$\mathbf{ad - bc \neq 0}$$

Characteristics

- Exists **only for square matrices**.
- Determinant must be **non-zero**.
- Multiplying a matrix by its inverse gives the **Identity Matrix**.

Conditions for Inverse

A matrix has an inverse only if:

- It is a **square matrix**.
- Its determinant is **not equal to zero**.

If

det(A) = 0

then the matrix is called a **Singular Matrix**, and **no inverse exists**.

Matrix Determinant

The **determinant** is a **single numerical value** calculated from a **square matrix**. It provides important information about the matrix, such as whether it is invertible and how it transforms area or volume.

The determinant of a matrix **A** is denoted by

det(A) or **|A|**

Determinant of a 2 × 2 Matrix

If

A =

a b

c d

then

det(A) = ad − bc

Characteristics

- Defined **only for square matrices**.
- Produces a **single scalar value**.
- Determines whether a matrix is invertible.
- Represents area scaling in **2D** and volume scaling in **3D**.

Geometric Meaning

- **2D:** Determinant represents the **area scaling factor**.
- **3D:** Determinant represents the **volume scaling factor**.

Examples

- **det = 2** → Area/Volume doubles.
- **det = 0.5** → Area/Volume becomes half.
- **det = 0** → Shape collapses (no area or volume).
- **det < 0** → Transformation includes a reflection (orientation changes).

Applications

- Finding the inverse of a matrix
- Solving systems of linear equations
- Checking linear independence
- Computer Graphics
- Machine Learning
- Engineering
- Data Science

Line, Plane, and Hyperplane

Dataset

In this project, the dataset consists of **students' scores in five subjects**.

Example

Student	Math	Science	English	Physics	Computer
S1	85	90	78	92	88
S2	80	85	82	88	84
S3	75	80	70	85	78
S4	90	95	88	94	92

Each subject represents one **feature (dimension)**.

Therefore

- **Number of Features = 5**
- **Dataset Dimension = 5D**

---

1. Line (1 Dimension)

A **line** is a one-dimensional (1D) geometric object that has only one direction.

If the dataset contained only **one feature**, such as **Math Score**, each student would lie on a line.

Example

Student	Math
S1	85
S2	80
S3	75

This is a **1-dimensional dataset**, so the data points lie on a **line**.

---

2. Plane (2 Dimensions)

A **plane** is a two-dimensional (2D) surface defined by two features.

If we select two subjects, for example:

- Math
- Science

each student can be represented as a point:

- S1 = (85, 90)
- S2 = (80, 85)
- S3 = (75, 80)

These points lie on a **2D plane**.

Example

Student	Math	Science
S1	85	90
S2	80	85
S3	75	80

Dimension = 2D

---

3. Hyperplane (3 or More Dimensions)

A **hyperplane** is a flat surface in **three or more dimensions** that separates data into different regions or classes.

Since our dataset contains **5 features**, each student is represented as:

(85, 90, 78, 92, 88)

This is a **5-dimensional point**.

A machine learning algorithm such as **Linear Discriminant Analysis (LDA)** or a **Linear Classifier** can use a **hyperplane** to separate students into classes, such as:

- Above Average
- Below Average

In this project, the hyperplane exists in **5-dimensional feature space**.

Dimension Progression

1 Feature → Line (1D)

2 Features → Plane (2D)

3 Features → Space (3D)

4+ Features → Hyperplane (nD)

Covariance

Covariance is a statistical measure that describes the **relationship between two variables**. It indicates whether two variables move **together** or in **opposite directions**.

- If one variable increases and the other also increases, the covariance is **positive**.
- If one variable increases while the other decreases, the covariance is **negative**.
- If there is no relationship, the covariance is approximately **zero**.

Formula

For two variables **X** and **Y**:

$\mathbf{Cov(X, Y) = \Sigma[(x_i - \bar{x})(y_i - \bar{y})] / (n - 1)}$

where:

- $\mathbf{x_i}$ = Individual value of variable **X**
- $\mathbf{y_i}$ = Individual value of variable **Y**
- $\mathbf{\bar{x}}$ = Mean of **X**
- $\mathbf{\bar{y}}$ = Mean of **Y**
- **n** = Number of observations

Interpretation

Positive Covariance

Cov(X, Y) > 0

- Variables move in the **same direction**.
- When **X increases**, **Y also tends to increase**.

Negative Covariance

Cov(X, Y) < 0

- Variables move in **opposite directions**.
- When **X increases**, **Y tends to decrease**.

Zero Covariance

Cov(X, Y) ≈ 0

- No significant **linear relationship** exists between the variables.

Characteristics

- Measures the **direction** of the relationship.
- Can be **positive, negative, or zero**.
- The magnitude depends on the units of measurement.
- Does **not** indicate the strength of the relationship.

Applications

- Finance
- Statistics
- Machine Learning
- PCA
- Data Science
- Robotics
- Forecasting
- Risk Analysis

Eigenvalues and Eigenvectors

What are Eigenvalues and Eigenvectors?

An **Eigenvector** is a **non-zero vector** whose direction remains unchanged when a linear transformation (matrix) is applied to it. Only its magnitude may change.

The factor by which the vector is scaled is called the **Eigenvalue**.

Mathematically

$Ax = \lambda x$

where:

- **A** = Square matrix
- **x** = Eigenvector
- **λ** = Eigenvalue

Eigenvector

Definition

An **Eigenvector** is a vector that **does not change its direction** after a matrix transformation. It may only be stretched, compressed, or reversed.

Characteristics

- Non-zero vector.
- Direction remains unchanged.
- Exists only for square matrices.
- Each eigenvector has a corresponding eigenvalue.

Eigenvalue

Definition

An **Eigenvalue** is a **scalar value** that indicates how much the corresponding eigenvector is stretched or compressed during the transformation.

Characteristics

- Scalar value.
- Represents the scaling factor.
- Can be positive, negative, or zero.
- Each eigenvalue corresponds to at least one eigenvector.

Characteristic Equation

Eigenvalues are found by solving the characteristic equation:

$$\det(A - \lambda I) = 0$$

where:

- **A** = Square matrix
- **I** = Identity matrix
- **λ** = Eigenvalue

After finding the eigenvalues, substitute each value into

$$(A - \lambda I)x = 0$$

to determine the corresponding eigenvectors.

Interpretation

Eigenvalue	Interpretation
Positive Eigenvalue	Vector keeps the same direction and is scaled.
Negative Eigenvalue	Vector reverses direction.

Eigenvalue	Interpretation
Zero Eigenvalue	Vector collapses to the zero vector.

Characteristics

- Defined only for **square matrices**.
- Eigenvectors are always **non-zero vectors**.
- A matrix may have multiple eigenvalues and eigenvectors.
- Eigenvalues may be real or complex.

Applications

- PCA
- Machine Learning
- Data Science
- Computer Vision
- Image Processing
- Face Recognition
- Signal Processing
- Quantum Mechanics
- Structural Engineering
- Network Analysis

LU Decomposition

What is LU Decomposition?

LU Decomposition is a matrix factorization technique in which a **square matrix** is decomposed into the product of two matrices:

- **L** = Lower Triangular Matrix
- **U** = Upper Triangular Matrix

Mathematically

A = LU

where:

- **A** = Original square matrix
- **L** = Lower triangular matrix
- **U** = Upper triangular matrix

Lower Triangular Matrix (L)

Definition

A **Lower Triangular Matrix** is a square matrix in which **all elements above the main diagonal are zero**.

Example

L =

1 0 0

2 1 0

3 4 1

Upper Triangular Matrix (U)

Definition

An **Upper Triangular Matrix** is a square matrix in which **all elements below the main diagonal are zero**.

Example

U =

2 3 1

0 5 4

0 0 6

LU Decomposition Formula

A = LU

where

L =

1 0 0

l_{21} 1 0

l_{31} l_{32} 1

and

U =

u_{11} u_{12} u_{13}

0 u_{22} u_{23}

0 0 u_{33}

Conditions

LU Decomposition is generally possible when:

- The matrix is **square**.
- The leading principal minors are **non-zero**.
- If these conditions are not satisfied, **Pivoting (PLU Decomposition)** is used.

Characteristics

- Decomposes a matrix into **Lower** and **Upper** triangular matrices.
- Simplifies solving systems of linear equations.
- Reduces repeated calculations.
- Efficient for solving multiple equations with the same coefficient matrix.

Applications

- Solving Linear Equations
- Matrix Inversion
- Determinant Calculation
- Numerical Analysis
- Machine Learning
- Engineering
- Computer Graphics
- Scientific Computing

Singular Value Decomposition (SVD) and Its Role in Dimensionality Reduction

---

What is Singular Value Decomposition (SVD)?

Singular Value Decomposition (SVD) is a matrix factorization technique that decomposes any matrix into three simpler matrices. It is one of the most powerful methods in **Linear Algebra**, **Machine Learning**, and **Data Science**.

For any matrix **A**,

$A = U\Sigma V^T$

where:

- **A** = Original matrix (**m × n**)
- **U** = Left Singular Vectors (Orthogonal Matrix)
- **Σ** = Diagonal Matrix containing Singular Values
- V^T = Transpose of Right Singular Vectors

Components of SVD

---

1. Left Singular Matrix (U)

Definition

The **U matrix** contains the **left singular vectors** and represents the directions of the original data.

Characteristics

- Orthogonal matrix.
- Columns are unit vectors.
- Represents the row space of the matrix.

---

2. Singular Value Matrix (Σ)

Definition

The **Σ (Sigma)** matrix is a diagonal matrix containing the **singular values** arranged in descending order.

Characteristics

- Diagonal matrix.
- Singular values are always non-negative.
- Larger singular values contain more important information.

---

3. Right Singular Matrix (V^T)

Definition

The **V^T matrix** contains the **right singular vectors**, representing the principal directions (features) of the data.

Characteristics

- Orthogonal matrix.
- Represents the column space of the matrix.

SVD Formula

$A = U\Sigma V^T$

Characteristics

- Works for **any matrix** (square or rectangular).
- Decomposes a matrix into three simpler matrices.
- Singular values are always non-negative.
- Singular values are ordered from largest to smallest.

Role of SVD in Dimensionality Reduction

Dimensionality reduction aims to reduce the number of features while preserving as much important information as possible.

Steps

1. Decompose the data matrix using SVD.
2. Sort singular values from largest to smallest.
3. Keep only the top **k** largest singular values.
4. Remove smaller singular values (less important information).
5. Reconstruct a lower-dimensional approximation of the original matrix.

Mathematically,

$$\mathbf{A \approx U_k \Sigma_k V_k^T}$$

where:

- **k** = Number of retained dimensions
- $\mathbf{U_k}$, $\mathbf{\Sigma_k}$, and $\mathbf{V_k^T}$ contain only the top **k** singular values and vectors.

Why SVD is Used for Dimensionality Reduction

- Removes redundant features.
- Reduces storage requirements.
- Speeds up machine learning algorithms.
- Reduces noise in the data.
- Preserves the most important information.

Applications

- Principal Component Analysis (PCA)
- Dimensionality Reduction
- Recommendation Systems

- Image Compression
- Machine Learning
- Data Compression

What is Principal Component Analysis (PCA)?

Principal Component Analysis (PCA) is a **dimensionality reduction** technique used to transform a large set of correlated features into a smaller set of **uncorrelated variables**, called **Principal Components**, while preserving as much information (variance) as possible.

PCA is widely used in **Machine Learning**, **Data Science**, **Computer Vision**, and **Statistics**.

Principal Components

Definition

A **Principal Component (PC)** is a new feature created by combining the original features. Each principal component captures the maximum possible variance in the data.

Characteristics

- Linear combination of original features.
- Uncorrelated (orthogonal) to each other.
- Ordered by the amount of variance they explain.
- The first principal component contains the highest variance.

PCA Process

Step 1

Standardize the dataset.

⬇

Step 2

Compute the covariance matrix.

⬇

Step 3

Calculate the eigenvalues and eigenvectors of the covariance matrix.

⬇

Step 4

Sort the eigenvalues in descending order.

⬇

Step 5

Select the top **k** principal components.

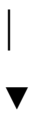
⬇

Step 6

Transform the original data into the new lower-dimensional space.

PCA Workflow

Original Dataset



Standardization

Covariance Matrix

Eigenvalues &

Eigenvectors

▼

Select Top-k

Components

|

▼

Reduced Dataset

PCA Formula

The transformed data is calculated as:

Z = XW

where:

- **X** = Original standardized data
- **W** = Matrix of selected eigenvectors (principal components)
- **Z** = Transformed lower-dimensional data

Characteristics

- Reduces the number of features.
- Preserves maximum variance.
- Produces uncorrelated principal components.
- Removes redundant information.
- Improves computational efficiency.

Applications

- Dimensionality Reduction
- Machine Learning
- Data Visualization

- Image Compression
- Face Recognition
- Pattern Recognition
- Computer Vision
- Bioinformatics
- Finance
- Signal Processing

What is Linear Discriminant Analysis (LDA)?

Linear Discriminant Analysis (LDA) is a **supervised dimensionality reduction** and **classification** technique that transforms data into a lower-dimensional space while maximizing the separation between different classes.

Unlike PCA, which maximizes variance, **LDA maximizes class separability**.

Objective of LDA

The main objective of LDA is to:

- Maximize the distance between different classes.
- Minimize the spread within the same class.
- Improve classification accuracy.

LDA Formula

LDA finds a projection vector **W** that maximizes the following ratio:

$J(W) = (W^T S^B W) / (W^T S^W W)$

where:

- **W** = Projection vector
- S^B = Between-Class Scatter Matrix

- S^W = Within-Class Scatter Matrix

Scatter Matrices

1. Within-Class Scatter Matrix (S^W)

Measures the variation of samples **within the same class**.

Objective

- Minimize S^W.
- Keep samples of the same class close together.

---

2. Between-Class Scatter Matrix (S^B)

Measures the variation **between different classes**.

Objective

- Maximize S^B.
- Increase the distance between different classes.

LDA Process

Step 1

Standardize the dataset (if necessary).

⬇

Step 2

Compute the **Within-Class Scatter Matrix (S^W)**.

⬇

Step 3

Compute the **Between-Class Scatter Matrix (S^B)**.

⬇

Step 4

Solve the generalized eigenvalue problem.

⬇

Step 5

Select the top discriminant vectors.

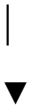
⬇

Step 6

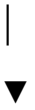
Project the original data onto the new feature space.

LDA Workflow

Original Dataset



Class Labels

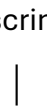


Compute S^W and S^B

Eigenvalue Problem

Select Top

Discriminant Vectors



Reduced Dataset

Characteristics

- Supervised learning technique.
- Uses class labels.
- Maximizes class separation.
- Reduces feature dimensions.
- Works best when classes are linearly separable.

Applications

- Classification
- Face Recognition
- Pattern Recognition
- Image Processing
- Medical Diagnosis
- Bioinformatics
- Finance
- Speech Recognition
- Machine Learning
- Data Science
